

Training-Free Probabilistic Time-Series Forecasting with Conformal Seasonal Pools

Valery Manokhin

Independent researcher

VALERY.MANOKHIN.2015@LIVE.RHUL.AC.UK

Editor: Ernst Ahlberg, Ulf Johansson, Henrik Boström, Alberto Carlevaro, Johan Hallberg Szabadváry and Lars Carlsson

Abstract

We propose Conformal Seasonal Pools (CSP), a training-free, conformal-style probabilistic time-series forecaster that mixes same-season empirical draws with signed residual draws around a seasonal-naive forecast. We evaluate it on a corpus of 232 datasets and 24,502 rolling-origin windows, which includes as a subset the six datasets on which DeepNPTS was introduced by [Rangapuram et al. \(2023\)](#).

Our headline result is about calibration, and it holds when the 860 forecast windows are grouped into the 58 series they come from rather than counted as independent. Grouping further, into the six datasets, leaves very little power: at $n = 6$ a signed-rank test cannot return a p below 0.03125 even under a clean sweep, so an effect can clear the conventional 5% level but cannot be established more stringently than that. We report that as a limit of the design, not as evidence against the result. DeepNPTS attains mean empirical coverage of 0.578 against a nominal 0.95, against 0.901 for CSP, and a mean scaled interval score (MSIS) of 48.1 against 29.4; treating each of the 232 datasets as the independent unit, the gap in $|\text{coverage} - 0.95|$ is significant at $p < 10^{-38}$ (paired Wilcoxon). The failure is more severe than aggregate coverage suggests: in **20.4%** of windows the DeepNPTS interval covers *none* of the H forecast horizons — the entire multi-step trajectory misses at every step simultaneously — against 1.5% for CSP. Such intervals fail the basic calibration desideratum of [Gneiting and Katzfuss \(2014\)](#): the stated level is not the level delivered, so any downstream rule that consumes it — reserve sizing, inventory buffers, alerting thresholds, risk limits — is calibrated against a probability that is wrong by more than a third.

On the distributional scores the margin is real but narrower, and we report it at the level the design supports. Across the full corpus CSP wins CRPS on 154 of 232 datasets and the Winkler interval score on 178; at dataset level both gaps are significant ($p \leq 10^{-5}$). On the six DeepNPTS datasets CSP has lower mean CRPS on five of six, improving by 21.4% on the mean of the per-dataset relative gaps — but with only six datasets that descriptive win does not reach significance under dataset-blocked inference ($p = 0.22$), and we do not claim otherwise.

Across the seven metrics we report, DeepNPTS leads on *none*, and is last on five. One methodological finding follows. DeepNPTS is trained on the ranked probability score, tuned on mean quantile loss, and evaluated on mean quantile loss — one functional form used three times — and that metric, being an average of pinball losses over interior quantiles, never charges an interval-containment penalty at all: it averages pinball loss over the nine interior quantiles $\{0.1, \dots, 0.9\}$, and an observation outside the 95% interval contributes only that same pinball loss, with no term keyed to whether the interval covered. A proper interval score charges $2/\alpha$ for exactly that event. Every metric we report that is sensitive to interval containment reverses the ordering. CSP achieves its results with no learned parameters and no training, at $819\times$ lower compute cost. We argue that training-free conformal samplers

should be mandatory baselines when evaluating learned non-parametric forecasters, and that a proper interval score should be reported alongside CRPS.

Keywords: Conformal prediction, probabilistic forecasting, time series, predictive distributions, CRPS, quantile loss, training-free baselines, seasonal naive, DeepNPTS.

1. Introduction

Probabilistic forecasting asks for a predictive distribution over future time-series values, not only a point forecast. This is the natural object in inventory planning, energy operations, transport, and other settings where decisions depend on uncertainty. Recent forecasting work has moved toward neural distributional models, including N-BEATS (Oreshkin et al., 2020), Temporal Fusion Transformers (Lim et al., 2021) and N-HITS (Challu et al., 2023); DeepNPTS (Rangapuram et al., 2023) is the corresponding non-parametric sampling baseline within that family and is the principal neural comparator we benchmark against. That shift is valuable, but it has also made it easy for low-compute baselines to be under-specified or omitted.

This paper studies a deliberately practical question: before training a neural sampler, how far can training-free empirical and conformal methods go? The question matters because many production forecasting systems need transparent baselines, fast reruns, cheap sensitivity checks, and failure modes that can be audited. A method whose intervals are well calibrated and whose runtime is in seconds is easier to deploy and audit than one that ships sharper intervals at the cost of nominal coverage and hours of training. Sharp intervals frequently hide severe undercoverage, failing the basic calibration desideratum of Gneiting and Katzfuss (2014) and posing tangible risks in safety- and decision-critical applications—health screening, autonomous driving, finance, and energy operations—where a prediction interval that systematically misses the truth is not merely inaccurate but actively misleading.

We evaluate that question against NPTS, SeasonalNPTS, and DeepNPTS. The main method family is Conformal Seasonal Pools (CSP), a simple sampler that mixes same-season empirical draws with signed residual draws around a seasonal naive forecast. CSP has no learned parameters, trains no neural network, and produces a full empirical predictive sample from which intervals, quantiles, CRPS, and quantile losses can be computed.

The claim is direct, and it is a claim about calibration first. On the six datasets where DeepNPTS was originally evaluated, **CSP-Repo** is far closer to nominal coverage, $|\text{coverage} - 0.95|$: mean coverage 0.949 vs 0.573, significant whether windows are treated as independent or blocked within series (paired Wilcoxon, $p = 3.5 \times 10^{-10}$ at the series level, $n = 58$). Across the whole corpus the same comparison at dataset level ($n = 232$) gives $p < 10^{-38}$. **CSP-Repo** also has lower mean CRPS on five of those six datasets; with six datasets that difference is descriptive rather than significant (dataset-blocked paired Wilcoxon, $p = 0.21875$), and the corpus-level CRPS result carries that claim instead. Across the seven metrics we report, **DeepNPTS leads on none**. It places fourth of six on CRPS rank, fifth on normalized MQL, and *last* on the remaining five — Winkler rank, MSIS, $|\text{coverage} - 0.95|$, log predictive density and PIT edge mass. **CSP-Repo** leads four of the seven (CRPS rank, MSIS, $|\text{coverage} - 0.95|$ and PIT edge mass); SeasonalNPTS leads Winkler rank and log predictive density; AdaptiveWindowMCI leads MQL. The pattern is not arbitrary: MQL averages pinball loss

over a quantile grid and, like CRPS, rewards a predictive distribution concentrated in the right place without separately charging for an interval that fails to contain the observation. The interval score charges $2/\alpha$ for exactly that, and the calibration diagnostics measure it directly. A method can therefore place well on the quantile-averaged losses while placing last on every metric that is sensitive to whether the interval actually covered.

We make no coverage guarantee: coverage is reported here as an empirical property of the sampler, not a theorem, and we do not claim CSP is calibrated at 0.95. It under-covers. Across the full corpus its mean coverage is 0.901 against a nominal 0.95 — a real shortfall, an order of magnitude smaller than DeepNPTS’s but a shortfall. On the six DeepNPTS datasets the mean of 0.949 conceals a per-dataset spread from 0.924 (wikipedia) to 0.988 (traffic), and the horizon-indexed residual lag L_h is what removes the long-horizon under-coverage that a single shared residual pool produces on non-seasonal series. The coverage gap is the most decision-critical: under the calibration-then-sharpness paradigm of [Gneiting and Katzfuss \(2014\)](#), a method whose nominal 95% interval misses the truth in over four windows in ten fails the basic calibration prerequisite and is inadmissible for any setting that takes the nominal level seriously. CSP runs in seconds where DeepNPTS runs in minutes ($819\times$ slower in the single-machine measurement of Appendix E). The result is not a benchmarking marginal: on its own evaluation suite a training-free seasonal residual sampler is better calibrated than the neural non-parametric forecaster at every blocking level the six-dataset design can resolve, and leads or ties it on every containment-sensitive score.

Contributions. The contributions of this paper are threefold. We propose *Conformal Seasonal Pools* (CSP), a training-free probabilistic time-series forecaster that mixes same-season empirical draws with signed conformal residual draws around a seasonal-naive forecast. We then run an audited rolling-origin benchmark on a corpus of 232 datasets and 24,502 windows, which includes as a subset the 6 datasets where DeepNPTS was originally evaluated by [Rangapuram et al. \(2023\)](#), organising the comparison around the calibration-then-sharpness protocol of [Gneiting and Katzfuss \(2014\)](#) and reporting per-dataset ranks, per-window paired Wilcoxon tests, the Winkler interval score, MSIS, log predictive density and randomized PIT alongside CRPS. Third, we show that CRPS alone is insufficient to rank probabilistic forecasters: a proper interval score reverses the ordering of two methods in our tables, and we recommend reporting one as standard.

2. Related Work

Probabilistic and neural forecasting. GluonTS provides a common toolkit for probabilistic forecasting experiments and reference implementations ([Alexandrov et al., 2020b](#)). Recent neural forecasting work includes Temporal Fusion Transformers for multi-horizon forecasting with covariates and interpretability ([Lim et al., 2021](#)), N-BEATS for deep univariate forecasting ([Oreshkin et al., 2020](#)), and N-HiTS for efficient long-horizon forecasting ([Challu et al., 2023](#)). DeepNPTS is the principal comparator we benchmark against: it revisits non-parametric forecasting through a global learned sampler, keeping the non-parametric goal of sampling from observed values while learning the sampling strategy from related series ([Rangapuram et al., 2023](#)).

Our benchmark does not claim to replace this literature. Instead, it asks whether a simpler sampler should be part of the standard comparison set. If a training-free method is competitive with a neural non-parametric sampler on proper scoring rules and dramatically faster, it changes the default burden of proof for more complex models.

Conformal prediction for time series. Conformal prediction provides distribution-free uncertainty sets under exchangeability (Vovk et al., 2005; Shafer and Vovk, 2008; Lei et al., 2018) and has been extended in several directions for dependent or shifting data. EnbPI constructs prediction intervals for dynamic time series without requiring full exchangeability (Xu and Xie, 2021, 2023). Adaptive conformal inference treats coverage under distribution shift as an online control problem (Gibbs and Candès, 2021). Conformalized quantile regression (Romano et al., 2019) adapts the conformal scaffold to quantile-based score functions. It is worth being explicit about the state of the art here: in the general time-series setting no conformal method delivers an unconditional finite-sample distribution-free coverage guarantee. The classical split-conformal bound (Vovk et al., 2005) requires exchangeability, which serial dependence in time series violates; EnbPI’s coverage holds approximately under stationarity-and-mixing conditions; and ACI (Gibbs and Candès, 2021) provides long-run online frequentist coverage rather than finite-sample marginal coverage. So all conformal time-series predictors operate on a spectrum from “approximate validity under stated conditions” to “asymptotic / online validity”; the absence of a hard finite-sample guarantee is the norm in this setting, not a CSP-specific deficit.

Conformal predictive distributions. A complementary line of work, conformal predictive distributions and conformal predictive systems, takes the conformal toolkit beyond intervals to produce calibrated predictive CDFs (Vovk et al., 2017, 2018b,a, 2020). Under exchangeability, a conformal predictive distribution $\hat{F}$ has the probability-integral-transform property $\hat{F}(Y) \sim \text{Uniform}(0, 1)$, which is a strictly stronger calibration criterion than empirical $1 - \alpha$ coverage at a single nominal level. The CSP method introduced here is an empirical-sample construction rather than a CPS in the strict sense.

CSP borrows the conformal habit of using empirical residuals, but it is a predictive-distribution sampler rather than only an interval constructor. The method uses signed seasonal residuals to preserve asymmetry and combines them with a same-season empirical pool to retain realistic distributional support.

Forecasting benchmarks and scoring. The M competitions, the Monash archive, and recent forecasting benchmark suites have made clear that conclusions depend on dataset diversity, scoring rules, and reproducibility choices (Makridakis et al., 2018, 2020, 2022; Godahewa et al., 2021; Zhang et al., 2024; Qiu et al., 2024). For probabilistic forecasts, strictly proper scoring rules and calibration-sharpness analysis are central (Gneiting and Raftery, 2007; Gneiting et al., 2007; Hersbach, 2000). We therefore use empirical CRPS as the primary distributional score and normalized mean quantile loss as a complementary quantile score.

3. Methods

Let $y_1, \dots, y_T$ be the observed history and let H be the forecast horizon. Every method returns an empirical sample matrix $S \in \mathbb{R}^{H \times B}$, with $B = 200$ samples in the audited run. Quantile forecasts and prediction intervals are derived from those samples.

Empirical pools and NPTS baselines. The simplest empirical forecaster samples directly from historical observations. A rolling empirical pool restricts the sample to recent history, while a seasonal empirical pool samples from the same seasonal phase as the target horizon. NPTS and SeasonalNPTS are related non-parametric samplers that weight historical observations by recency and, for SeasonalNPTS, by seasonal phase.

These methods are robust and fast, but they can fail in predictable ways. A full-history empirical pool ignores level shifts and seasonality. A rolling pool adapts to level shifts but loses seasonal structure. A seasonal pool can become too small for yearly or short histories. NPTS-style recency weighting helps, but it does not by itself recenter forecasts around the latest seasonal level.

Conformal residual samplers. A seasonal naive forecast uses $\mu_h = y_{T+h-m}$ when a seasonal period m is available, falling back to the latest observation when necessary. Residual conformal samplers compute calibration residuals and then generate samples of the form $\mu_h + r$, where r is drawn from an empirical residual pool. In this benchmark the residual pool is signed rather than absolute when the method is used to generate a full distribution.

The residual view captures local level and recent seasonal changes better than raw empirical sampling. Its weakness is that residual pools can be horizon-blind or too symmetric if implemented only as absolute interval scores.

Conformal Seasonal Pools. CSP combines the two views. For each horizon h , it constructs a predictive sample from a mixture

$$\hat{F}_h = w_h \hat{F}_h^{\text{season}} + (1 - w_h) \hat{F}_h^{\text{resid}},$$

and the B columns of row h of the sample matrix S are drawn from $\hat{F}_h$. Here $\hat{F}_h^{\text{season}}$ samples observed values from the same seasonal position as $T + h$ with exponential recency weights, and $\hat{F}_h^{\text{resid}}$ samples $\mu_h + r$ for r a signed residual from the calibration window. The weight w_h is set adaptively: it is 0 when seasonality is absent ($m \leq 1$), reduced when fewer than three seasonal cycles are observed, and w otherwise, so the sampler degenerates to the pure residual component exactly when the seasonal pool is undefined. Both public implementations we benchmark use this adaptive rule, and we report one row per implementation. Algorithm 1 states the procedure formally.

AdaptiveWindowMCI. CSP takes the seasonal period m from dataset metadata and pools over the whole history. **AdaptiveWindowMCI** relaxes both. It estimates the period from the data as $\hat{m} = \arg \max_{2 \leq \tau \leq \min(200, T/3)} |\hat{\rho}(\tau)|$, the lag of the largest absolute autocorrelation; when that peak falls below 0.3 it does *not* declare the series aperiodic but falls back to the supplied metadata period, so the method removes CSP’s reliance on metadata only where the data support an alternative. It then truncates the series to a recent window $W = \min(T, \max(50, 2\hat{m}))$ and runs the CSP construction on that window alone, so *both* components — the seasonal pool and the conformal residual calibration — see only

the truncated history. The mixture weight follows CSP’s own adaptive rule applied to the truncated window: 0 when $\hat{m} \leq 1$, $\min(w, 0.3)$ when fewer than three cycles fit in W , and w otherwise. It is an ablation of CSP’s two structural inputs — supplied seasonality and unrestricted history — and it tests whether either is doing real work.

Conformal validity remarks. The name “Conformal Seasonal Pools” is intended to indicate that the residual component is *motivated by* split-conformal calibration, not that the full mixture inherits a finite-sample coverage proof. The signed-residual pool plays the role of a split-conformal calibration set in the spirit of Papadopoulos et al. (2002), who introduced inductive (split) conformal prediction for regression, but the construction we use—signed residuals, an empirical centred interval extracted from the predictive sample, and a calibration window taken from the most recent half of training history under residual exchangeability that holds only approximately for time series—differs in details from the canonical split-conformal interval whose finite-sample bound is proved in that reference. We therefore make no formal coverage claim, even for the residual-only sub-sampler, and we make no formal coverage claim for the full CSP mixture, where the seasonal empirical pool contributes to sharpness without a coverage proof. The strong empirical performance of split-conformal-style constructions in the time-series setting, despite the formal exchangeability violation, has recently received a theoretical explanation. Oliveira et al. (2024) prove that split conformal is approximately valid for non-exchangeable data with quantifiable degradation; Barber and Pananjady (2025) sharpen this by bounding the coverage loss of split conformal under temporal dependence with a “switch coefficient” that quantifies the exchangeability violation, and show this characterization is sharp over the class of stationary β -mixing processes. Together these results provide post-hoc justification for the design choice we make: split-conformal residual calibration is empirically effective for time series even though the canonical finite-sample guarantee no longer applies, and the gap between effective and guaranteed coverage is now characterised in the literature. As discussed in Section 2 on conformal prediction for time series, this is the norm rather than the exception in the time-series CP literature: EnbPI (Xu and Xie, 2021), ACI (Gibbs and Candès, 2021), AgACI (Zaffran et al., 2022), and CTSF (Stankevičiūtė et al., 2021) all rely on conditions (stationarity-and-mixing, asymptotic / online behaviour, per-series exchangeability) that themselves cannot be checked from finite data, and none delivers an unconditional finite-sample coverage guarantee in the general case. CSP is in good company in foregoing such a guarantee. Coverage is reported throughout this paper as an empirical finite-sample property, and is one of the metrics the method is judged on rather than one it is guaranteed to satisfy.

Mixture weights and coverage. At the values used in this paper ($w_h \in \{0, 0.3, 0.5\}$) at least half of every CSP draw comes from the conformally-motivated residual pool, and when $w_h = 0$ the mixture is the residual-only sampler exactly. We do not state a closed-form bound on mixture coverage in terms of component coverages, since the prediction interval is extracted from the mixture sample as a data-dependent quantile and does not decompose linearly across components; Section 5.3 reports the empirical coverage achieved instead.

The default $w = 0.5$ is not tuned per dataset, so it is worth showing what the choice costs. Table 1 sweeps w on taxi30 over the same 120 windows, holding everything else fixed. The two endpoints are the ablations of interest: $w = 0$ is the residual-only sampler

Input : history $y_{1:T}$, horizon H , seasonal period m , sample budget B , calibration fraction ρ , recency rate λ , pool weight w , minimum pool size $\kappa = 8$

Output: predictive sample matrix $S \in \mathbb{R}^{H \times B}$

$T_{\text{cal}} \leftarrow \min(T, \max(\lfloor \rho T \rfloor, m + 1))$, $c \leftarrow y_{T-T_{\text{cal}}+1:T}$

if $m \leq 1$ **then**
 | $w_{\text{eff}} \leftarrow 0$ *(seasonal pool undefined)*
else if $T/m < 3$ **then**
 | $w_{\text{eff}} \leftarrow \min(w, 0.3)$ *(too few cycles)*
else
 | $w_{\text{eff}} \leftarrow w$
end
 $B_{\text{pool}} \leftarrow \lfloor B w_{\text{eff}} \rfloor$

Function $\text{ResidualPool}(L)$:
 | **if** $T_{\text{cal}} - L \geq \kappa$ **then return** $\{c_{t+L} - c_t : 1 \leq t \leq T_{\text{cal}} - L\}$ *(calibration window)*
 | **if** $T - L \geq \kappa$ **then return** $\{y_{t+L} - y_t : 1 \leq t \leq T - L\}$ *(full history, same lag)*
 | **for** $\ell \leftarrow L - 1$ **to** 1 **do**
 | | **if** $T - \ell \geq \kappa$ **then return** $\sqrt{L/\ell} \{y_{t+\ell} - y_t : 1 \leq t \leq T - \ell\}$ *(shortened lag, rescaled)*
 | **end**
 | **return** $\sqrt{L} \{y_{t+1} - y_t : 1 \leq t < T\}$ *(scaled one-step differences)*

for $h \leftarrow 1$ **to** H **do**
 | $b_{\text{pool}} \leftarrow B_{\text{pool}}$, $b_{\text{conf}} \leftarrow B - B_{\text{pool}}$ *(reset every horizon)* $L_h \leftarrow m \cdot \lceil h/m \rceil$ *(horizon-indexed lag;*
 | $L_h = m$ *whenever* $h \leq m$) $\mathcal{R}_h \leftarrow \text{ResidualPool}(L_h)$ **if** $m \leq 1$ **then** $\mu_h \leftarrow y_T$
 | **else**
 | | $i \leftarrow T + h - m$; **while** $i \geq T$ **do** $i \leftarrow i - m$
 | | **if** $1 \leq i \leq T$ **then** $\mu_h \leftarrow y_i$
 | | **else** $\mu_h \leftarrow y_T$
 | **end**
 | $\mathcal{I}_h \leftarrow \{t \leq T : t \equiv T + h \pmod{m}\}$, $\pi_t \propto \exp(-\lambda(\max \mathcal{I}_h - t))$ **if** $b_{\text{pool}} > 0$ **and** $m > 1$ **and**
 | $|\mathcal{I}_h| \geq 2$ **then**
 | | draw b_{pool} values from $\{y_t\}_{t \in \mathcal{I}_h}$ by inverse CDF on π
 | **else**
 | | $b_{\text{conf}} \leftarrow b_{\text{conf}} + b_{\text{pool}}$ *(pool too thin at this h; budget falls through)*
 | **end**
 | draw b_{conf} values $\mu_h + r$, $r \sim \text{Uniform}(\mathcal{R}_h)$ $S_{h,\cdot} \leftarrow$ concatenation of the two draws
end
return S

Algorithm 1: Conformal Seasonal Pools, as implemented in `csp-forecaster` v0.2.0 (Manokhin, 2026) on package defaults ($w = 0.5$, $\lambda = 0.01$, $\rho = 0.5$, $\kappa = 8$, `residual_mode="h_step"`, `decay_unit="step"`, `orientation=False`), with the sample budget raised from the package default of 100 to $B = 200$ for every sample-based method in this benchmark. `RESIDUALPOOL` returns the calibration-window pool whenever that window yields at least κ residuals, which holds on the large majority of the corpus; the remaining branches are starvation guards for short histories at long lags, where a one-element pool would collapse the predictive distribution to a point mass. They are not merely hypothetical: on `m3_yearly` ($T = 14$, $m = 1$, $H = 6$) the calibration window yields at most $6 < \kappa$ residuals, so every horizon takes the full-history branch. Prediction intervals are plain empirical quantiles of S .

Table 1: Pool-weight ablation on **taxi30** ($H = 24$, $m = 48$, 120 windows), all other settings fixed. ^r residual-only; ^s seasonal-pool-only; **bold** is the package default used throughout the paper.

w	Coverage	$ \cdot - 0.95 $	MSIS	Scaled CRPS
0.00 ^r	0.926	0.024	8.32	0.772
0.10	0.940	0.010	7.41	0.750
0.25	0.943	0.007	7.02	0.722
0.50	0.934	0.016	6.96	0.697
0.75	0.902	0.048	7.19	0.697
1.00 ^s	0.762	0.188	10.83	0.726

and $w = 1$ the seasonal-pool-only sampler. Removing the residual component entirely is the one setting that fails badly — coverage falls to 0.762 and MSIS rises to 10.83, worse than the residual-only endpoint and worse than the mixture — which is the concrete evidence that the conformal residual draw, not the seasonal pool, is what carries calibration. The interior of the range is flat: every $w \in [0.1, 0.75]$ holds coverage within 0.05 of nominal, and scaled CRPS varies by under 0.054 across that span. The default is therefore a safe rather than a tuned choice; $w = 0.25$ would have been marginally better calibrated on this dataset and $w = 0.5$ marginally sharper, and we did not select between them on test scores. This is a single-dataset probe, not a sensitivity study across the corpus, and we do not claim it generalises.

4. Experimental Design

The audited evaluation covers six datasets from the GluonTS reference suite—electricity, exchange_rate, solar_energy, taxi, traffic, and wikipedia—the evaluation set used by [Rangapuram et al. \(2023\)](#) to introduce DeepNPTS, at the horizons and window counts given in their Table 1. Taxi is their 30-minute panel (**taxi_30min**, 1214 series, 24×56). Forecast origins for the other five datasets are taken by stepping back from the end of each series in strides of H , which is a rolling-origin scheme of the same form but not identical to the GluonTS canonical splits. We compare the two publicly available CSP implementations—**CSP-Repo** ([Manokhin, 2026](#)) and **CSP-Nixtla** ([Nixtla, 2026](#))—and **AdaptiveWindowMCI**, a second training-free sampler we propose, against three external comparators from the GluonTS NPTS family (NPTS, SeasonalNPTS, DeepNPTS) ([Alexandrov et al., 2020a](#)). Ranks are reported over the six benchmarked methods. The benchmark produces 860 forecast records per method (5,160 result rows in total) under a rolling-origin protocol with a fixed horizon per dataset and identical windows across methods, so per-(dataset, series, window) comparisons are paired. Pairing is by window, not by random draw: seeding differs by driver, as [Appendix A](#) sets out. Pairing is carried by a **window_uid** assigned in the corpus build; the released **series_id** labels are truncated, so 155 label keys stand for 781 distinct physical windows, so all analysis keys on **window_uid** rather than on the label. Recovered pairing is validated by requiring that the seasonal-naive scale implied independently

Table 2: Distributional-score and cost summary across the 6 datasets and 860 forecast records per method. Lower ranks are better; coverage targets 0.95.

Method	CRPS rank	Winkler rank	MQL rank	Coverage	$ \cdot - 0.95 $	MSIS
CSP-Repo	2.17	2.83	2.83	0.949	0.021	18.2
CSP-Nixtla	2.33	3.33	3.00	0.875	0.096	25.9
AdaptiveWindowMCI	4.17	3.83	2.67	0.719	0.232	28.3
DeepNPTS	3.83	4.33	4.33	0.573	0.377	46.7
SeasonalNPTS	3.00	2.67	3.83	0.893	0.063	24.7
NPTS	5.50	4.00	4.33	0.948	0.025	26.8

by CRPS and by the interval score agree across all six methods within every window; 21 collided windows for which that check cannot be completed are excluded from the corpus rather than carried with an unverified identity, and the build fails if any remaining window disagrees (Appendix E).

We report empirical CRPS, empirical 95% interval coverage, interval width, log predictive density (Gaussian KDE, Silverman bandwidth), randomized PIT, and audited method wall time. Log predictive density is sensitive to the KDE bandwidth rule and is reported as a secondary diagnostic; no ranking in this paper depends on it. Normalized mean quantile loss is computed on the nine-point grid $\tau \in \{0.1, 0.2, \dots, 0.9\}$. Rangapuram et al. (2023) report MQL on the finer grid 0.05 to 0.95; we retain the coarser grid across all methods so every method is scored identically, and note that the difference is not in DeepNPTS’s disfavour, since the omitted levels are the two most extreme on each side, precisely where its intervals are widest relative to the observation. MQL is reported for continuity with prior work but is not used to rank methods: it is a quantile-averaged loss, not a proper scoring rule for the full predictive distribution, and Section 7 gives a concrete case where it is actively misleading. Lower CRPS and normalized MQL are better. Coverage is descriptive because the benchmark ranks methods by distributional scores rather than by interval coverage alone.

Runtime is extracted from logs. For non-DeepNPTS methods, wall time is the logged elapsed time for each parallel method evaluation. For DeepNPTS, wall time is the elapsed time from each train-start line to the corresponding saved-result line. Dataset download/loading and final summary writing are excluded.

5. Results

5.1. Aggregate distributional scores

CSP-Repo achieves the best mean CRPS rank (2.17 over the six benchmarked methods), ahead of CSP-Nixtla at 2.33, SeasonalNPTS at 3.00, DeepNPTS at 3.83, AdaptiveWindowMCI at 4.17 and NPTS at 5.50. On normalized MQL the ordering differs sharply — AdaptiveWindowMCI ranks best at 2.67 despite placing fifth on CRPS and fifth on calibration, a discrepancy taken up in Section 7.

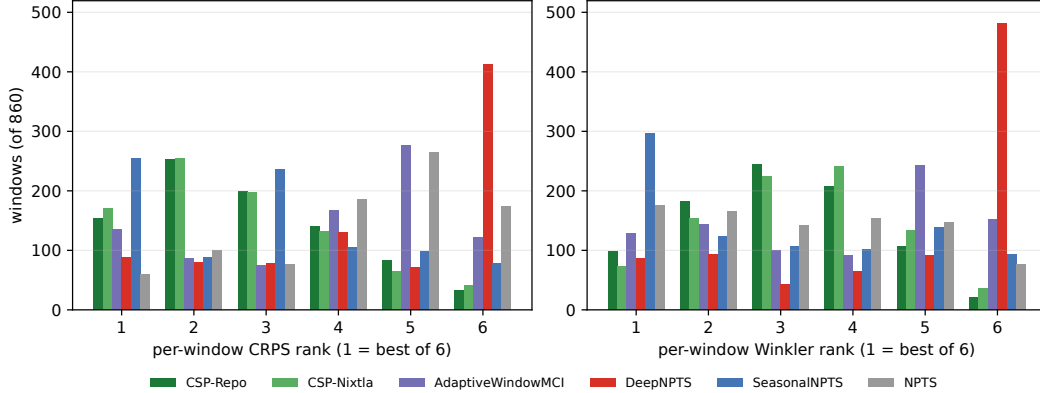


Figure 1: Per-window rank distribution (860 tier-1 forecast windows, six methods). Bars count the windows in which each method lands in each rank band; green is CSP-Repo, red DeepNPTS. CSP variants concentrate in the top three ranks on both metrics.

Table 3: Per-window rank-finish distribution over the 860 tier-1 forecast windows. Each window is ranked 1 (best) to 6 (worst) within itself; cells give the *proportion* of a method’s windows landing in each rank band, so rows sum to 1.

Method	CRPS rank					Winkler rank				
	R1	R2	R3	R4	R5+	R1	R2	R3	R4	R5+
CSP-Repo	0.18	0.29	0.23	0.16	0.13	0.11	0.21	0.28	0.24	0.15
CSP-Nixtla	0.20	0.30	0.23	0.15	0.12	0.08	0.18	0.26	0.28	0.20
AdaptiveWindowMCI	0.16	0.10	0.09	0.19	0.46	0.15	0.17	0.12	0.11	0.46
DeepNPTS	0.10	0.09	0.09	0.15	0.56	0.10	0.11	0.05	0.07	0.67
SeasonalNPTS	0.30	0.10	0.27	0.12	0.20	0.35	0.14	0.12	0.12	0.27
NPTS	0.07	0.12	0.09	0.22	0.51	0.20	0.19	0.17	0.18	0.26

5.2. Rank distribution

Figure 1 shows the per-window rank distribution for each method on CRPS and on the Winkler interval score; Table 3 reports the underlying proportions. Each window is ranked within itself across the benchmarked methods. The contrast between the two panels is the point: DeepNPTS’s CRPS ranks are spread across the middle of the distribution, while its Winkler ranks concentrate at the bottom. Ranking on CRPS alone would place it mid-field.

5.3. Calibration analysis at the 95% level

The accepted paradigm for evaluating probabilistic forecasts, set out by Gneiting et al. (2007) and consolidated in the review of Gneiting and Katzfuss (2014), is to *maximize sharpness subject to calibration*. Calibration is the prerequisite: a probabilistic forecast that is not calibrated cannot be improved by sharpening it, because sharpness without calibration is just narrower wrongness. Under this paradigm a forecaster is first checked for calibration, and only then are its sharpness scores admissible.

CSP-Repo achieves mean empirical coverage of 0.949 against the nominal 0.95 target; CSP-Nixtla reaches 0.875. We stress that *larger coverage is not better calibration*: a method sitting at 1.00 is not better calibrated than one at 0.94. All calibration comparisons in this paper are therefore made on the absolute deviation $|\text{coverage} - 0.95|$, paired per window, and Table 9 reports them on that basis.

Read that way the picture is not a clean sweep, and we report it as it is. CSP-Repo is decisively better calibrated than DeepNPTS (0.059 vs 0.409 mean absolute per-window deviation, $p < 10^{-16}$) and than CSP-Nixtla and SeasonalNPTS. It is *worse* calibrated than plain NPTS on the six-dataset subset (0.059 vs 0.049 per window, $p = 0.021$). NPTS attains near-nominal coverage by producing very wide intervals: it is the widest-interval method in the suite and simultaneously the weakest on CRPS (mean rank 5.50 of six). This is calibration without sharpness. Under the maximize-sharpness-subject-to-calibration paradigm of Gneiting and Raftery (2007) the two failure modes are distinct and both matter: NPTS satisfies the calibration prerequisite and then wastes it, while DeepNPTS fails the prerequisite outright. CSP-Repo beats NPTS on per-window CRPS on 647 of 860 windows ($p < 10^{-16}$) while giving up 0.010 of mean absolute coverage deviation, which is the trade we would make.

SeasonalNPTS deserves its own note, since it is the closest non-CSP competitor. CSP-Repo is better calibrated than SeasonalNPTS (0.059 vs 0.106 mean absolute per-window deviation, $p = 0.031$ dataset-blocked) and ranks ahead on CRPS, MQL and MSIS; SeasonalNPTS edges it on Winkler rank (2.67 vs 2.83). The CRPS margin is the weakest of these: 2.17 against 3.00 on rank, and SeasonalNPTS is beaten on 453 of 860 paired windows, but that margin is not significant ($p = 0.139$ per window, $p = 0.5625$ dataset-blocked), and on dataset-mean CRPS it is SeasonalNPTS that leads on three of the six. On cost the two are in the same class — both are training-free samplers costing milliseconds per record, not the $819\times$ separating CSP from DeepNPTS. We do not put a figure on that ratio: the SeasonalNPTS timing carried in Table 11 is inherited from an earlier run whose raw transcript was not retained, and only the CSP-Repo and DeepNPTS timings rest on a recorded measurement. The runtime argument in this paper is an argument about *training-free samplers as a class* versus a trained neural sampler. DeepNPTS achieves 0.573 mean coverage, with across-dataset standard deviation 0.292.

A method whose nominal 95% prediction interval contains the truth in only about 57% of cases fails the basic calibration desideratum of Gneiting and Katzfuss (2014): the interval it reports is not the interval it delivers, however competitive its quantile-loss number.

In safety- and decision-critical applications the practical consequences are stark. A 0.95 nominal interval that covers in only $\sim 57\%$ of cases is not a usable decision input: any

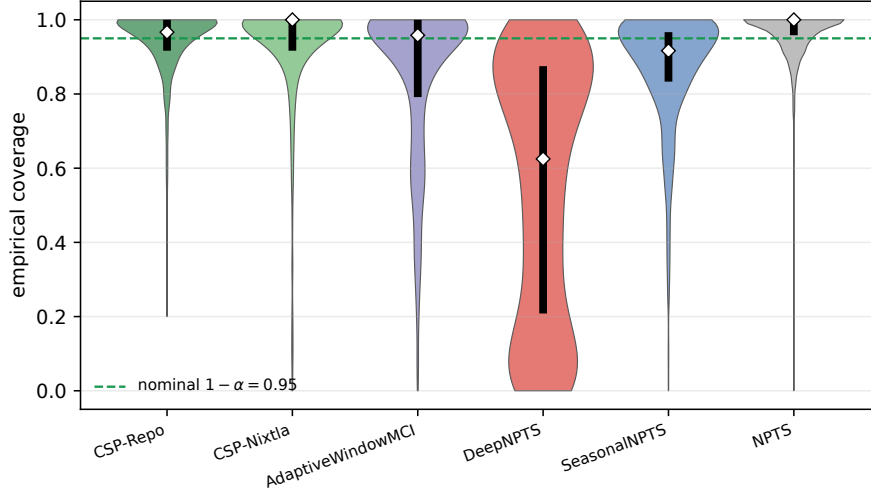


Figure 2: Per-window empirical coverage distribution across all 860 forecast windows. Each violin shows the distribution of empirical coverage at the per-(dataset, series, window) level; white diamonds mark the median, black bars the inter-quartile range. The dashed green line marks the nominal $1 - \alpha = 0.95$ target. CSP variants concentrate near 1.0 with thin lower tails; DeepNPTS is bimodal with substantial mass near zero. The diagnostic shows DeepNPTS fails calibration both by being systematically biased low (median 0.625) and by being highly variable across windows (standard deviation 0.346).

downstream rule that treats the stated level as the true level—reserve sizing, inventory buffers, alerting thresholds, risk limits—is calibrated against a probability that is wrong by more than a third. Regulated settings make the point sharpest, since they specify the level explicitly: the Basel market-risk framework, for instance, is built on expected shortfall at a 97.5% one-tailed level ([Basel Committee on Banking Supervision, 2019](#)), and a model whose nominal level is off by this margin could not be used to compute it. The gap between nominal 0.95 and empirical 0.573 is not a benchmark detail; it is a barrier to using the interval as anything other than a point forecast with decoration.

Figure 2 shows the per-window coverage distribution. DeepNPTS fails calibration in two compounding ways: its mean coverage is biased low (0.573 versus the 0.95 nominal target), and its window-to-window spread is large (standard deviation 0.346). The decisive number is the lower decile of the DeepNPTS distribution: 0.04. Concretely, in the worst 10% of the six-dataset subset windows the DeepNPTS 95% interval contains the realised value at almost none of the H horizons. Across the corpus this is not a tail curiosity but a fifth of all windows: 20.4% of DeepNPTS windows are *total trajectory misses*, with the interval containing the truth at none of the H steps, against 1.5% for CSP.

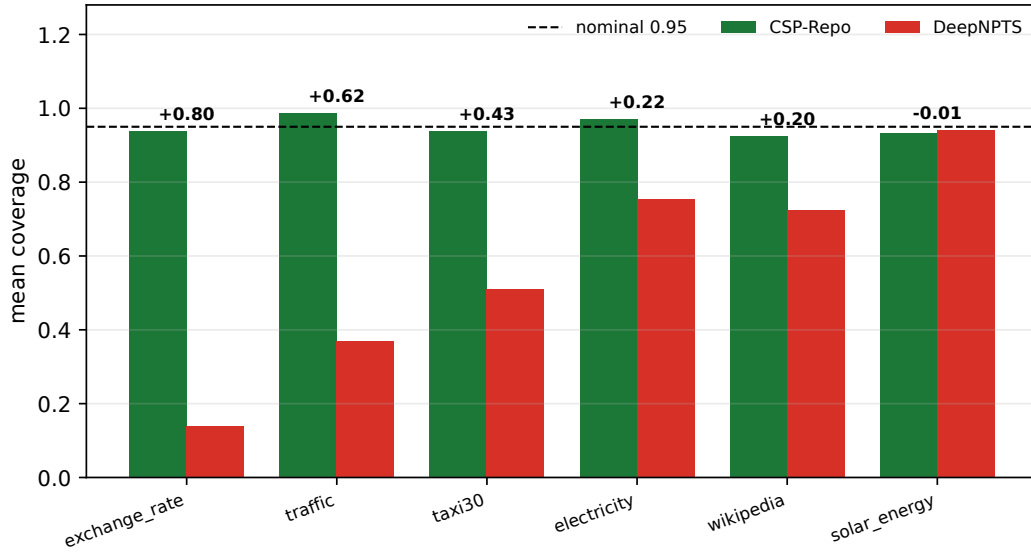


Figure 3: Per-dataset mean coverage on the six DeepNPTS datasets (860 windows), sorted by gap. **CSP-Repo** (green) covers at or above 0.95 on four of six and is nearer nominal than DeepNPTS (red) on five of six; the exception is *solar_energy*, where DeepNPTS is marginally nearer (0.94 vs 0.93). Labels above each pair give the CSP-DeepNPTS coverage difference in absolute coverage units.

Table 4: Dataset-level head-to-head wins (6 datasets). Each cell reports CRPS wins out of 6; lower CRPS wins.

Method	vs NPTS	vs SeasonalNPTS	vs DeepNPTS
CSP-Repo	6/6	3/6	5/6
CSP-Nixtla	6/6	3/6	5/6

5.4. Head-to-head comparisons

CSP-Repo beats DeepNPTS on 5/6 datasets by CRPS and 5/6 by MQL; CSP-Nixtla shows the same pattern. Table 4 reports the full head-to-head matrix. Against NPTS the CSP variants win 6/6 datasets on CRPS and 4/6 on MQL; against SeasonalNPTS they win 3/6 on both.

The CRPS-MQL gap on the DeepNPTS comparison is a known limitation of normalized MQL rather than a substantive draw. MQL is the average pinball loss over the nine fixed quantile levels $\{0.1, 0.2, \dots, 0.9\}$, so it only sees the body of the predictive distribution and is blind to behaviour in the tails $\tau < 0.1$ and $\tau > 0.9$. CRPS, by contrast, integrates over the entire predictive CDF and so penalises tail miscalibration directly. DeepNPTS’s failure mode is exactly tail-driven — median per-window coverage 0.625, lower decile 0.042 at the 0.95 nominal level (Section 5.3) — which is what an MQL on $\{0.1, \dots, 0.9\}$ is built to under-weight and what CRPS is built to capture. The headline metric CRPS is therefore the appropriate score for comparing miscalibration profiles, and the MQL agreement is a secondary check.

5.5. Paired significance tests and absolute scores

Forecast windows are nested within series and series within datasets, so windows are not independent replicates. Treating the 860 tier-1 windows as 860 independent pairs is pseudoreplication and overstates precision, sometimes by many orders of magnitude. We therefore declare the independence unit before testing: **the dataset is the primary unit** ($n = 6$ on the DeepNPTS subset, $n = 232$ on the full corpus), the series is a secondary blocking level, and per-window tests are reported only as descriptive sensitivity analyses. All tests are paired Wilcoxon signed-rank.

Table 5 shows what this costs. Both comparisons still hold when windows are grouped by series. Neither is detectable when they are grouped by dataset, but that is a statement about $n = 6$, not about the methods: at that sample size the test has almost no resolving power left. We report it rather than quoting the window-level figure as though it had 860 degrees of freedom.

With $n = 6$ the dataset-blocked signed-rank test cannot return a p below 0.03125 even under a perfect sweep, so neither dataset-level entry is evidence about effect size; the series-blocked column is the informative one. We report the dataset level anyway because it is the most conservative unit available and we would rather show its limits than omit it.

Table 5: Tier-1 **CSP-Repo** vs DeepNPTS, testing the same two comparisons three times while changing what counts as one independent observation: each window, each series, or each dataset. Both advantages survive grouping by series. Neither is detectable when grouped by dataset, because $n = 6$ is too small: the signed-rank test cannot return a p below 0.03125 at that size no matter how one-sided the result.

	per window $n = 860$	series-blocked $n = 58$	dataset-blocked $n = 6$
CRPS	2.7×10^{-42}	8.8×10^{-3}	0.219
$ \text{cov} - 0.95 $	$< 10^{-16}$	3.5×10^{-10}	0.0625

What holds. On the full corpus, at dataset level ($n = 232$): CSP-Repo beats DeepNPTS on $|\text{coverage} - 0.95|$ at $p < 10^{-38}$, on MSIS at $p < 10^{-29}$, on the Winkler score at $p < 10^{-17}$, and on CRPS at $p = 3.4 \times 10^{-6}$. It wins CRPS on 154 of 232 datasets and Winkler on 178. Both CSP variants are ahead of all three NPTS-family methods on mean CRPS rank and mean MQL rank at every cell of Table 2.

What does not. Against SeasonalNPTS there is no CRPS advantage on this subset at any level ($p = 0.139$ per window, $p = 0.5625$ dataset-blocked; SeasonalNPTS leads on three of six dataset means). NPTS is slightly better calibrated than **CSP-Repo** on the subset (0.049 vs 0.059 per window) — by producing much wider intervals, and at the cost of losing 647 of 860 windows on CRPS. We report these rather than select around them.

5.6. Runtime

Runtime is where the gap is largest. Every method was timed on the same machine, single-threaded, on CPU, and every runtime figure in this paper is reconstructed from per-record timings that ship with the release: a `runtime_s` column for the training-free methods and a per-training-group `group_train_s` column for DeepNPTS, joined to the corpus on (dataset, series, window, method). Coverage is 97.7% of corpus records for the training-free methods and 100% for DeepNPTS. Because the per-record cost distribution is strongly right-skewed, we quote the median for the training-free methods; the mean-based ratio is $486\times$ and is reported in Appendix F. DeepNPTS consumes **37.6 hours** of CPU time to train 1,205 networks, one global model per (dataset, window) group, summed across four parallel shards, at 5.52 s per forecast record. **CSP-Repo** needs 6.75 ms per record and trains nothing. **DeepNPTS is 819 $\times$ slower.** That ratio depends on the training protocol, so we state the alternative explicitly rather than only in passing. Retraining per rolling-origin window is what a rolling-origin evaluation requires: a single model fitted once per dataset over all origins would let observations from later windows inform forecasts made at earlier ones. Granting DeepNPTS that cheaper protocol anyway, as an upper bound on how favourable the comparison could be made, it needs 1,064 ms per record and remains $158\times$ slower (Appendix F; derived from the measured per-record cost by scaling training with the number of models fitted, which overstates the saving and so bounds DeepNPTS’s advantage from above). The cost conclusion is therefore robust to the protocol choice: the

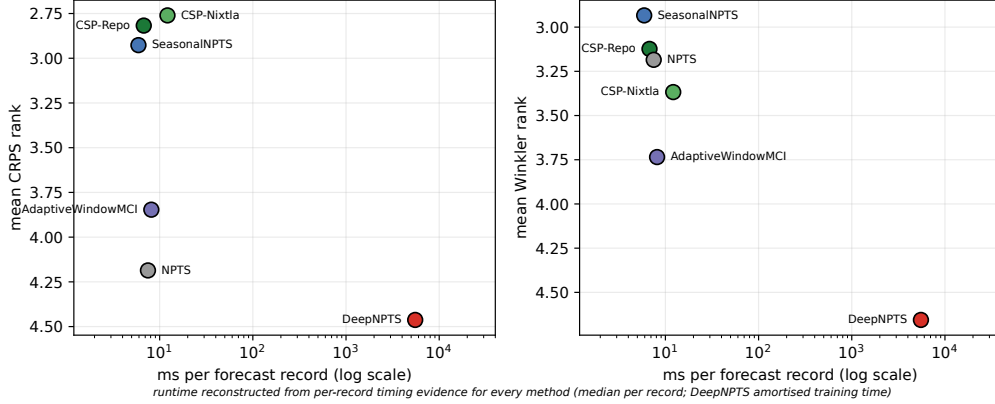


Figure 4: Distributional score vs. cost. Lower mean CRPS rank is better; the x -axis is log-scaled wall time. **CSP-Repo** and **CSP-Nixtla** sit in the upper-left training-free zone; **DeepNPTS** is dominated on rank and isolated by two orders of magnitude on cost while also failing the coverage target.

gap is two orders of magnitude under the protocol most favourable to **DeepNPTS**, and nearly three under the one its own evaluation implies. Figure 4 places these numbers in an score-versus-cost view: every training-free method sits in the upper-left zone (low CRPS rank, sub-second runtime), and **DeepNPTS** is the only point in the upper-right — worse on rank, and nearly three orders of magnitude slower. Full per-method wall-time numbers are reported in Appendix F.

6. The Full Corpus

Evaluating on a comparator’s own benchmark invites the objection that the result may not generalise. We therefore repeat the entire protocol on a corpus of 232 datasets and 24,502 rolling-origin windows, drawn from the Monash archive, the GluonTS and LOTSA collections, the chronos parquet corpus and assorted public sources, at frequencies from five-minutely to yearly. The six datasets of Section 5 are a labelled subset of it. Every method sees identical windows and identical targets; random draws are not matched across methods (Appendix A). Windows on which any method failed are removed from all methods, so no method is scored on data another could not attempt.

DeepNPTS leads on none of the seven metrics. It places fourth of six on CRPS rank, fifth on normalized MQL, and *last* on the remaining five — Winkler rank, MSIS, $|\text{coverage} - 0.95|$, log predictive density and PIT edge mass. **CSP-Repo** leads four of the seven; **SeasonalNPTS** leads Winkler rank and log predictive density; **AdaptiveWindowMCI** leads MQL. The pattern is not arbitrary. MQL averages pinball loss over a quantile grid and, like CRPS, rewards a predictive distribution concentrated in the right place without separately charging for an interval that fails to contain the observation. The interval score charges $2/\alpha$ for exactly that, and the calibration diagnostics measure it directly. A method

Table 6: Full corpus: 232 datasets (246 harvests, averaged within dataset), 24,502 windows. Ranks are recomputed over the six methods shown. Lower is better for ranks, MSIS, MQL and $|\text{coverage} - 0.95|$; higher for log density. Coverage targets 0.95.

Method	CRPS rank	Wink. rank	MSIS	MQL	Coverage	$ \cdot - 0.95 $	Log dens.
CSP-Repo	2.81	2.78	29.4	0.542	0.901	0.066	-5.25
CSP-Nixtla	2.93	3.04	36.3	0.543	0.872	0.092	-5.46
AdaptiveWindowMCI	3.45	3.93	35.9	0.448	0.756	0.195	-6.29
DeepNPTS	3.70	4.56	48.1	0.524	0.578	0.372	-8.13
SeasonalNPTS	3.71	3.59	43.3	0.643	0.763	0.193	-5.86
NPTS	4.39	3.10	41.5	0.617	0.857	0.102	-5.52

can therefore place well on the quantile-averaged losses while placing last on every metric sensitive to whether the interval actually covered.

The CRPS margin is modest; the interval-score margin is not. CSP-Repo wins CRPS on 154 of 232 datasets and on 56.5% of individual windows. On the Winkler score it wins 178 of 232 datasets and 63.7% of windows, and on MSIS 190 of 232. CRPS integrates over the whole predictive distribution and is forgiving of a distribution that is sharp in the wrong place; the interval score is not. A benchmark reporting CRPS alone would rank DeepNPTS mid-table. We therefore recommend reporting a proper interval score alongside CRPS as standard practice.

The calibration gap is the dominant effect. DeepNPTS covers 0.578 against the nominal 0.95 across the corpus, against 0.901 for CSP-Repo; the dataset-weighted mean $|\text{coverage} - 0.95|$ is 0.372 against 0.066. Two regimes drive it. On *non-seasonal* series (5,025 windows) DeepNPTS covers 0.258 with MSIS 87.5, against 0.814 and 37.2 for CSP-Repo; in the largest regime, $m < H \leq 5m$ (12,011 windows), it covers 0.583 with MSIS 78.3, against 0.908 and 57.2. In both it is the worst method tested. The aggregate also hides the tail: in **20.4%** of all windows the DeepNPTS 95% interval contains the realised value at *none* of the H horizons — a total trajectory miss — against **1.5%** for CSP-Repo. One window in five.

7. What Works, What Does Not, and Where

The CSP variants succeed because their two components correct different failure modes of empirical sampling: the signed-residual component recenters the predictive distribution around the latest seasonal level, while the seasonal pool keeps samples on realistic support. Using the horizon-indexed lag $L_h = m \lceil h/m \rceil$ rather than a single lag m matters only for $h > m$, and reduces to the original construction whenever $h \leq m$; for non-seasonal series ($m = 1$) it gives $L_h = h$, which is what removes the multi-step under-coverage. NPTS shows that pure recency-weighted empirical sampling is too coarse on hourly data (CRPS rank 5.50 of six); SeasonalNPTS is the closest non-CSP comparator from the NPTS family (rank 3.00, coverage 0.893).

DeepNPTS, despite being the most parameter-rich method in the comparator set, ranks fourth of six on CRPS and is the worst-calibrated method in the entire suite: mean coverage 0.573 against the nominal 0.95 target, 47.2% of randomized PIT values in the extreme tails, and a log predictive density of -5.94 against -2.10 for CSP-Repo. CSP-Repo improves on it by 21.4% CRPS on the mean of the per-dataset relative gaps and wins five of six on mean CRPS, losing only solar_energy; with $n = 6$ that margin is descriptive, not significant (Table 5).

Pooled means and ranks disagree on MQL, and we report both. A reviewer of an earlier version noted a contradiction in our MQL statements, and it is real. On the six datasets, DeepNPTS attains a *lower pooled mean* normalized MQL than CSP-Repo (0.578 against 0.587) while placing *worse on mean MQL rank* (4.33 against 2.83). The two summaries answer different questions: the pooled mean is dominated by the few datasets whose normalized MQL is largest, whereas the rank is computed within each dataset and so weights all six equally. This is a Simpson-style reversal, and neither number is wrong. We therefore never write that one method “outperforms” another on MQL without saying which summary is meant, and the same caution applies to any pooled mean over datasets of very different scale. This comparison also supplies the clearest argument for not ranking by normalized MQL. AdaptiveWindowMCI attains the *best* mean MQL in the table (0.441, against 0.587 for CSP-Repo) while sitting fifth of six on CRPS rank and fifth on $|\text{coverage} - 0.95|$ at 0.232. A quantile-averaged loss can therefore be won by a method that is neither the sharpest nor close to nominal. CRPS, the interval scores, log density and PIT all place the methods where the calibration evidence says they belong, and DeepNPTS — fifth on MQL but last on all five containment-sensitive metrics — shows the same gap from the other direction.

8. Why the Original Evaluation Could Not Detect This

The result above is not a contradiction of Rangapuram et al. (2023). On the metric that paper reports, DeepNPTS is competitive on its own six datasets — its mean normalized MQL of 0.578 sits between SeasonalNPTS (0.602) and NPTS (0.566), and it beats both CSP variants on two of the six datasets. The disagreement is not about that number; it is about what that metric can see.

Three features of the original protocol are relevant, all stated in that paper. First, DeepNPTS is *trained* on the ranked probability score, “a discrete version of the continuous ranked probability score”, which is a sum of quantile losses. Second, its hyperparameters are *tuned* using “the mean quantile loss as the criteria”. Third, forecasts are *evaluated* using “the mean of quantile losses evaluated at different quantile levels ranging from 0.05 to 0.95”. Training objective, model-selection criterion and reported metric are the same functional form. The competitors in that comparison optimise something else — DeepAR and SQF maximise a likelihood, AutoETS and AutoARIMA fit by maximum likelihood, Seasonal-Naive optimises nothing — and are then scored on DeepNPTS’s objective.

This matters because a quantile-averaged loss cannot, by construction, isolate whether a central prediction interval contains the observation. Averaging pinball loss across $\alpha = 0.05$ to 0.95 lets accurate behaviour near the median offset failure in the tails. Every metric we report that *is* sensitive to interval containment — the Winkler score, MSIS, $|\text{coverage} - 0.95|$,

log predictive density and PIT edge mass — places DeepNPTS last of the six methods, while the quantile-averaged loss ranks it above only NPTS.

The original paper does address calibration, and its treatment is the narrowest part of the evaluation: a single figure on a single dataset (Traffic), compared against DeepAR, SQF and Seas.NPTS. On that same dataset we measure DeepNPTS at 0.369 mean empirical coverage against a nominal 0.95 — below the Seas.NPTS it is shown outperforming (0.905) and well below plain NPTS (0.949). The calibration plot used there reports $\text{coverage}(\alpha)$ averaged over quantile levels, series and time, which is a weaker diagnostic than per-window coverage at a fixed level and is subject to the same tail-masking.

We draw a methodological conclusion rather than a critical one. A benchmark that reports a single quantile-averaged score cannot support a claim about calibration, and a model evaluated on its own training objective should be read with that in mind. Reporting a proper interval score alongside CRPS, and coverage at a stated nominal level rather than averaged across levels, costs nothing and would have surfaced this at the time.

9. Limitations and Scope

Scope of the comparison set. The benchmark is scoped to non-parametric samplers: the NPTS family (NPTS, SeasonalNPTS, DeepNPTS) plus several training-free conformal and empirical samplers including CSP. We deliberately did not include classical distributional baselines (ETS, ARIMA, or TBATS with bootstrapped residuals), full-distribution conformal time-series methods built on parametric forecasters (e.g. EnbPI applied as a sampler), or other neural probabilistic forecasters such as DeepAR, Temporal Fusion Transformers, or N-HITS. The goal of this paper is the head-to-head between training-free conformal sampling and DeepNPTS as the strongest published non-parametric neural sampler, and the choice of comparator set reflects that goal. The pattern of intervals systematically too narrow to attain nominal coverage that this paper documents for DeepNPTS is consistent with the broader prediction-interval failure documented across the M4 competition submissions (Grushka-Cockayne and Jose, 2020).

Benchmark size and selection. Five of the six datasets contribute 10 series; exchange_rate contributes 8, which is its full size in the GluonTS release. Each series carries 5–7 rolling-origin windows, except taxi, where we follow the 56 windows specified in Table 1 of Rangapuram et al. (2023), giving 860 paired forecast records per method overall. The series-count cap is set by the requirement that the DeepNPTS comparator be retrained from scratch for every (dataset, window) group — one global model fitted across that group’s series, as its own protocol specifies — and remain auditable on a single workstation; full-series GluonTS configurations of the same datasets contain hundreds of series each and would not have been DeepNPTS-auditable in the available compute budget. The wall-clock comparison is a local CPU measurement; GPU hardware and different DeepNPTS training schedules will narrow the runtime gap, but two orders of magnitude is a wide enough margin that the qualitative conclusion is robust to those choices. The coverage gap is the more important comparison and does not depend on hardware.

Statistical and methodological caveats. The split-conformal residual pool used by CSP assumes exchangeability of calibration and test residuals, which is only approximately

satisfied for non-stationary series; [Xu and Xie \(2021\)](#) and [Gibbs and Candès \(2021\)](#) provide drop-in extensions that we expect would further improve coverage on the most non-stationary datasets in the suite. The head-to-head against DeepNPTS is confined to its own evaluation suite (electricity, exchange_rate, solar_energy, taxi, traffic, wikipedia), which is the fair comparison on its design ground; the 232-dataset corpus extends the CSP evidence but is not part of that head-to-head.

Scope of the construction. CSP is deliberately minimal, and its boundaries are worth stating precisely. The sampler treats forecast horizons independently within each predictive sample, so the columns of the sample matrix are marginal draws rather than coherent trajectories; applications needing joint multi-horizon dependence admit a coupling step such as a block bootstrap, which the construction accommodates without modification. The signed-residual pool is uniform over the calibration window; studentizing it by a rolling seasonal MAD is a natural extension for heteroskedastic or strongly drifting series. Every result reported here uses a single fixed setting of the sample budget $B = 200$, the recency rate λ and the calibration fraction ρ , held constant across all 232 datasets — so these are untuned defaults, not a tuned best case.

10. Conclusion

On the six datasets where DeepNPTS was originally benchmarked, a simple training-free conformal sampler is far closer to nominal coverage (mean 0.949 vs 0.573; series-blocked paired Wilcoxon $p = 3.5 \times 10^{-10}$) and has lower mean CRPS on five of the six, while running 819× faster on CPU in a single-machine measurement. Across a corpus of 232 datasets and 24,502 windows the same conclusion holds and strengthens: CSP wins CRPS on 154 of 232 datasets and the Winkler interval score on 178, DeepNPTS leads none of the seven metrics we report, and a total trajectory miss — an interval containing the truth at none of the H horizons — occurs in 20.4% of DeepNPTS windows against 1.5% for CSP. The coverage gap is the one that matters most for use: under the calibration-then-sharpness protocol of [Gneiting and Katzfuss \(2014\)](#), a method whose nominal 95% interval misses the truth in over four windows in ten fails the calibration prerequisite, and its stated level cannot be relied on by any rule that consumes it.

We frame this as evidence that training-free seasonal residual samplers should be included as mandatory baselines when evaluating learned non-parametric forecasters, and that future probabilistic-forecasting benchmarks should report empirical coverage at the nominal level alongside any sharpness score. The result is specific to the audited DeepNPTS configuration on this benchmark and is not a general claim about neural probabilistic forecasting.

Acknowledgments

The authors thank the maintainers of the GluonTS toolkit for the public datasets and the reference DeepNPTS implementation used in this benchmark.

Appendix A. Implementation Details

This appendix documents the parameter choices made for every method in the audited suite.

CSP-Repo and CSP-Nixtla. CSP-Repo is `csp-forecaster` v0.2.0 (Manokhin, 2026), available at github.com/valeman/csp-forecaster. CSP-Nixtla is `statsforecast.models.ConformalSeasonalPool` (Nixtla, 2026). DeepNPTS is the GluonTS reference implementation (Alexandrov et al., 2020a); NPTS and SeasonalNPTS are our own re-implementations of the NPTS sampling rule, described in Appendix E.

Algorithm 1 states the construction as implemented in `csp-forecaster` v0.2.0; CSP-Nixtla is `statsforecast.models.ConformalSeasonalPool` with `variant="adaptive"`. Both are run on package defaults with no per-dataset tuning. The weight rule ($w_{\text{eff}} \in \{0, 0.3, 0.5\}$) is fixed across all datasets. Hyperparameters are the `csp-forecaster` package defaults, not tuned here: $\rho = 0.5$ (calibration fraction), $\lambda = 0.01$ (recency rate, aged in *steps*), $w = 0.5$ (pool weight), $B = 200$ predictive samples per forecast record (matched across all training-free methods). The finite-sample orientation correction available in the package is left *off*; enabling it raises coverage (for example $0.919 \rightarrow 0.928$ on wikipedia) at unchanged CRPS, so the figures reported here are the conservative ones.

Rolling-origin protocol. Every dataset uses rolling origins with stride equal to its forecast horizon H , so forecast targets never overlap, and the same window indices are used across all methods so comparisons are paired. Window counts and horizons vary across the corpus: on the six Tier-1 datasets each series carries 5–7 windows at $H = 24$ (hourly) or $H = 30$ (daily), except taxi, which follows the 56 windows of Rangapuram et al. (2023). Across the full 232-dataset corpus the horizon is whatever the source archive specifies for that dataset — H ranges over $\{6, 8, 12, 18, 24, 30, 48, 96, \dots\}$ and per-series window counts range from 1 to 56. Table 7 gives the composition.

DeepNPTS. Run via `gluonts.torch.model.deep_npts.DeepNPTSEstimator` with 50 epochs, batch size 32, two hidden layers of 40 units, learning rate 10^{-4} , 100 batches/epoch, context multiplier 4 (capped at 512 steps), 200 inference samples. No early stopping, no per-dataset tuning. CPU only.

Other methods. DeepNPTS: the GluonTS reference implementation. NPTS and SeasonalNPTS are *our own re-implementations* of the NPTS sampling rule rather than the GluonTS predictor: each draws from the empirical distribution of past values with exponential recency weights at rate 0.005, SeasonalNPTS restricting the pool to the same seasonal phase and NPTS drawing from the whole history. They differ from `gluonts.model.npts.NPTSPredictor` in two ways that matter: the decay rate is fixed at 0.005 rather than the package default, and the seasonal phase is taken from the supplied period m rather than derived from calendar time features. We report them as re-implementations, not as the reference predictor. Seeding is deterministic but not uniform across drivers, and we state the three regimes rather than a single rule. The main sweep derives a seed per record as $\text{CRC32}(\text{dataset}|\text{series}|\text{window}|\text{method}|20260424) \bmod 2^{32}$; because the method name enters the hash, each method draws its own random numbers, so

records are individually reproducible but draws are *not* shared across methods. The taxi driver instead derives one seed per window and reuses it for both CSP implementations, which does share the draw between those two. DeepNPTS is seeded once globally per run, as its training loop is not driven by our per-record rule. Consequently the benchmark is paired *by window* — every method is scored on identical inputs and identical targets — but we make no claim of matched random draws across methods.

Per-dataset preprocessing. No normalization, gap-filling, or frequency conversion. Series shorter than $H + 10$ observations are filtered out.

Appendix B. Dataset Suite

Table 7 gives the composition of the audited corpus by harvest source. Tier-1 is the DeepNPTS evaluation set and is a labelled subset of the corpus rather than a separate suite; row totals are asserted equal to the corpus totals when the table is generated.

Table 7: Audited dataset suite. Records are forecast records per method.

Source	Provenance	Datasets	Series	Windows
Tier-1	GluonTS reference (DeepNPTS’s evaluation set)	6	58	860
arrow	LOTSa / BuildingsBench arrow	109	1998	10,220
misc	miscellaneous public series	13	55	275
pq	Monash / M-competition parquet	44	1547	7,228
s3	S3 four-way harvest	23	100	650
tier2	M3 / M4 / M5 / NN5 / tourism family	16	155	335
tsf	Monash native .tsf	35	1112	4,934
Total		246	5025	24,502

Appendix C. Per-Method Absolute Scores

Table 8 reports the per-method mean of within-(dataset, series, window) normalized CRPS, normalized MQL, and empirical coverage at the 0.95 nominal level.

Appendix D. Per-Window Paired Significance Tests

Table 9 reports the full battery of per-window paired Wilcoxon signed-rank tests on within-(dataset, series, window) normalized CRPS and MQL (each method’s score divided by the cross-method median for that window) and on coverage. The headline numbers are reported in Section 5.5; this appendix records the comparison set in full.

Appendix E. Environment and Run Provenance

Every number in this paper was produced with the execution environment below; the versions listed are those of the result-producing run, not of whatever host last regenerated the release artifacts. Seeding is per record, not global: each (dataset, series, window, method)

Table 8: Absolute-score summary across the 6 datasets. CRPS is reported as a within-dataset ratio to the cross-method median to control for the wide range of dataset scales. MQL is the normalized mean quantile loss. Coverage targets 0.95.

Method	CRPS rank	Wink. rank	CRPS (rel.)	MSIS	MQL	Coverage	$ \cdot - 0.95 $	Log dens.
CSP-Repo	2.17	2.83	0.907	18.2	0.587	0.949	0.021	-2.10
CSP-Nixtla	2.33	3.33	0.935	25.9	0.586	0.875	0.096	-2.53
AdaptiveWindowMCI	4.17	3.83	1.090	28.3	0.441	0.719	0.232	-3.94
DeepNPTS	3.83	4.33	1.283	46.7	0.578	0.573	0.377	-5.94
SeasonalNPTS	3.00	2.67	1.000	24.7	0.602	0.893	0.063	-1.93
NPTS	5.50	4.00	1.490	26.8	0.566	0.948	0.025	-2.61

Table 9: Per-window paired Wilcoxon signed-rank tests over the 860 tier-1 windows. Calibration is tested as $|\text{coverage} - 0.95|$ and accuracy as CRPS; MQL is not tested here. Per-window tests are secondary to the dataset-level analysis of Table 5.

CSP-Repo vs	mean $ \text{cov} - 0.95 $		p (calib.)	CRPS wins	p (CRPS)
	CSP	other			
CSP-Nixtla	0.059	0.076	0.003	411/860	0.387
DeepNPTS	0.059	0.409	$< 10^{-16}$	631/860	$< 10^{-16}$
SeasonalNPTS	0.059	0.106	$< 10^{-16}$	453/860	0.139
NPTS	0.059	0.049	0.021	647/860	$< 10^{-16}$

Table 10: Pinned environment and protocol constants.

Component	Version / value	Role
<code>Python</code>	3.11.15	result-producing run
<code>numpy</code>	2.0.2	result-producing run
<code>pandas</code>	2.2.2	result-producing run
<code>scipy</code>	1.15.3	result-producing run
<code>torch</code>	2.11.0	result-producing run
<code>gluonts</code>	0.16.2	result-producing run
<code>csp-forecaster</code>	0.2.0	result-producing run
<code>statsforecast</code>	2.1.1	result-producing run
Master seed	20260424	all methods
Per-record seed	CRC32(ds series window method master)	derived, not global
Predictive samples	200	all sample-based methods
DeepNPTS epochs	50	100 batches/epoch, CPU, 1 thread
Windows (Tier-1)	860	6 datasets
Windows (Tier-2)	335	16 datasets
Methods	6	identical windows; draws not matched

tuple derives its own seed by the CRC32 rule of Appendix A, so a re-run reproduces every training-free record exactly. Draws are not shared across methods, and the pairing this benchmark relies on is pairing by window rather than by draw (Appendix A). DeepNPTS retrains a network per window group and therefore carries run-to-run variance which we did not quantify: every DeepNPTS number in this paper comes from a single seeded run, and we make no claim about its dispersion across seeds. Replicating the DeepNPTS sweep across seeds would sharpen the comparison and is left to future work.

Appendix F. Per-Method Wall Time

Table 11 reports wall time per method, reconstructed by `reconstruct_runtimes.py` from the per-record timing columns retained in the released sweep files. No entry is a hand-declared constant. We report both the median and the mean per-record cost because the distribution is right-skewed: a small number of long series dominate the mean, so the median is the fairer central estimate and the mean is the conservative one. On the mean basis DeepNPTS is $486\times$ slower rather than $819\times$; both bases leave the cost conclusion two to three orders of magnitude apart. Dataset download, parsing and result writing are excluded; the measurement covers model construction and prediction only, single-threaded and on CPU for every method.

Table 11: Method wall time, reconstructed from per-record timing evidence. Dataset download/loading and final summary writing are excluded.

Method	ms / record		Total CPU	Models trained	Slowdown (median)
	median	mean			
CSP-Repo	6.75	11.37	4.5 min	0	819×
CSP-Nixtla	12.13	20.64	8.2 min	0	455×
AdaptiveWindowMCI	8.14	14.75	5.9 min	0	679×
DeepNPTS	5,524.90	5,524.90	37.6 h	1,205	1×
SeasonalNPTS	5.92	9.87	3.9 min	0	933×
NPTS	7.49	15.51	6.2 min	0	738×

Reconstructed from the per-record `runtime_s` and `group_train_s` columns retained in the released sweep files, joined to the corpus on (dataset, series, window, method); coverage is at least 97.7% of corpus records for every method. No entry is a hand-declared constant. The slowdown column uses the median CSP cost; on the mean basis the DeepNPTS factor is 486×

References

- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C. Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, Lorenzo Stella, Ali Caner Türkmen, and Yuyang Wang. GluonTS: Probabilistic and neural time series modeling in Python. *Journal of Machine Learning Research*, 21(116):1–6, 2020a. URL <https://github.com/awsmlabs/gluonts>.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C. Maddix, Syama Sundar Rangapuram, David Salinas, Jasper Schulz, Lorenzo Stella, Ali Caner Turkmen, and Yuyang Wang. GluonTS: Probabilistic and neural time series modeling in Python. *Journal of Machine Learning Research*, 21(116):1–6, 2020b. URL <http://jmlr.org/papers/v21/19-820.html>.
- Rina Foygel Barber and Ashwin Pananjady. Predictive inference for time series: Why is split conformal effective despite temporal dependence?, 2025. URL <https://arxiv.org/abs/2510.02471>.
- Basel Committee on Banking Supervision. MAR33: Internal models approach: backtesting and P&L attribution requirements. Basel Framework, Bank for International Settlements, 2019. https://www.bis.org/basel_framework/chapter/MAR/33.htm.
- Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza, Max Mergenthaler-Canseco, and Artur Dubrawski. N-HiTS: Neural hierarchical interpolation for time series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 6989–6997, 2023. doi: 10.1609/aaai.v37i6.25854. URL <https://doi.org/10.1609/aaai.v37i6.25854>.
- Isaac Gibbs and Emmanuel J. Candès. Adaptive conformal inference under distribution shift, 2021. URL <https://arxiv.org/abs/2106.00170>.
- Tilmann Gneiting and Matthias Katzfuss. Probabilistic forecasting. *Annual Review of Statistics and Its Application*, 1:125–151, 2014. doi: 10.1146/annurev-statistics-062713-085831. URL <https://doi.org/10.1146/annurev-statistics-062713-085831>.
- Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437. URL <https://doi.org/10.1198/016214506000001437>.
- Tilmann Gneiting, Fadoua Balabdaoui, and Adrian E. Raftery. Probabilistic forecasts, calibration and sharpness. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(2):243–268, 2007. doi: 10.1111/j.1467-9868.2007.00587.x. URL <https://doi.org/10.1111/j.1467-9868.2007.00587.x>.
- Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. Monash time series forecasting archive, 2021. URL <https://arxiv.org/abs/2105.06643>.
- Yael Grushka-Cockayne and Victor Richmond R. Jose. Combining prediction intervals in the M4 competition. *International Journal of Forecasting*, 36(1):178–185, 2020. doi: 10.1016/j.ijforecast.2019.04.015. URL <https://doi.org/10.1016/j.ijforecast.2019.04.015>.
- Hans Hersbach. Decomposition of the continuous ranked probability score for ensemble prediction systems. *Weather and Forecasting*, 15(5):559–570, 2000. doi: 10.1175/1520-0434(2000)015<0559:DOTCRP>2.0.CO;2. URL [https://doi.org/10.1175/1520-0434\(2000\)015<0559:DOTCRP>2.0.CO;2](https://doi.org/10.1175/1520-0434(2000)015<0559:DOTCRP>2.0.CO;2).
- Jing Lei, Max G’Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111, 2018. doi: 10.1080/01621459.2017.1307116. URL <https://doi.org/10.1080/01621459.2017.1307116>.

- Bryan Lim, Sercan Ö. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4): 1748–1764, 2021. doi: 10.1016/j.ijforecast.2021.03.012. URL <https://doi.org/10.1016/j.ijforecast.2021.03.012>.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4):802–808, 2018. doi: 10.1016/j.ijforecast.2018.06.001. URL <https://doi.org/10.1016/j.ijforecast.2018.06.001>.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1):54–74, 2020. doi: 10.1016/j.ijforecast.2019.04.014. URL <https://doi.org/10.1016/j.ijforecast.2019.04.014>.
- Spyros Makridakis, Evangelos Spiliotis, Vassilios Assimakopoulos, Zhi Chen, Anil Gaba, Ilia Tsetlin, and Robert L. Winkler. The M5 uncertainty competition: Results, findings and conclusions. *International Journal of Forecasting*, 38(4):1365–1385, 2022. doi: 10.1016/j.ijforecast.2021.10.009. URL <https://doi.org/10.1016/j.ijforecast.2021.10.009>.
- Valery Manokhin. csp-forecaster: Conformal seasonal pools for training-free probabilistic time-series forecasting, 2026. URL <https://github.com/valeman/csp-forecaster>. Reference implementation, tag v0.2.0, commit 3063568875eff09123b4afce8b9703247444d561; package defaults used throughout.
- Nixtla. statsforecast: Lightning-fast forecasting with statistical and econometric models, 2026. URL <https://github.com/Nixtla/statsforecast>. ConformalSeasonalPool; documentation at <https://nixtlaverse.nixtla.io/statsforecast/docs/models/conformalseasonalpool.html>.
- Roberto I. Oliveira, Paulo Orenstein, Thiago Ramos, and João Vitor Romano. Split conformal prediction and non-exchangeable data. *Journal of Machine Learning Research*, 25(225):1–47, 2024. URL <http://jmlr.org/papers/v25/23-1553.html>.
- Boris N. Oreshkin, Dmitri Carpo, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlecqn4YwB>.
- Harris Papadopoulos, Kostas Proedrou, Vladimir Vovk, and Alex Gammerman. Inductive confidence machines for regression. In *Machine Learning: ECML 2002*, volume 2430 of *Lecture Notes in Computer Science*, pages 345–356. Springer, 2002. doi: 10.1007/3-540-36755-1_29. URL https://doi.org/10.1007/3-540-36755-1_29.
- Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. TFB: Towards comprehensive and fair benchmarking of time series forecasting methods. *Proceedings of the VLDB Endowment*, 17(9): 2363–2377, 2024. doi: 10.14778/3681954.3682003. URL <https://doi.org/10.14778/3681954.3682003>.
- Syama Sundar Rangapuram, Jan Gasthaus, Lorenzo Stella, Valentin Flunkert, David Salinas, Yuyang Wang, and Tim Januschowski. Deep non-parametric time series forecaster, 2023. URL <https://arxiv.org/abs/2312.14657>.
- Yaniv Romano, Evan Patterson, and Emmanuel J. Candès. Conformalized quantile regression. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://papers.nips.cc/paper/2019/hash/5103c3584b063c431bd1268e9b5e76fb-Abstract.html>.
- Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9:371–421, 2008. URL <http://jmlr.org/papers/v9/shafer08a.html>.
- Kamilė Stankevičiūtė, Ahmed M. Alaa, and Mihaela van der Schaar. Conformal time-series forecasting. In *Advances in Neural Information Processing Systems*, vol-

- ume 34, pages 6216–6228, 2021. URL <https://papers.nips.cc/paper/2021/hash/312f1ba2a72318edaaa995a67835fad5-Abstract.html>.
- Vladimir Vovk, Alex Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005. doi: 10.1007/b106715. URL <https://doi.org/10.1007/b106715>.
- Vladimir Vovk, Jieli Shen, Valery Manokhin, and Min-ge Xie. Nonparametric predictive distributions based on conformal prediction. In *Proceedings of the Sixth Workshop on Conformal and Probabilistic Prediction and Applications*, volume 60 of *Proceedings of Machine Learning Research*, pages 82–102. PMLR, 2017. URL <https://proceedings.mlr.press/v60/vovk17a.html>.
- Vladimir Vovk, Ilia Nouretdinov, Valery Manokhin, and Alex Gammerman. Cross-conformal predictive distributions. In *Proceedings of the Seventh Workshop on Conformal and Probabilistic Prediction and Applications*, volume 91 of *Proceedings of Machine Learning Research*, pages 37–51. PMLR, 2018a. URL <https://proceedings.mlr.press/v91/vovk18a.html>.
- Vladimir Vovk, Ilia Nouretdinov, Valery Manokhin, and Alex Gammerman. Conformal predictive distributions with kernels. In *Braverman Readings in Machine Learning. Key Ideas from Inception to Current State*, volume 11100 of *Lecture Notes in Computer Science*, pages 103–121. Springer, 2018b. doi: 10.1007/978-3-319-99492-5_4. URL https://doi.org/10.1007/978-3-319-99492-5_4.
- Vladimir Vovk, Ilia Nouretdinov, Valery Manokhin, and Alex Gammerman. Computationally efficient versions of conformal predictive distributions. *Neurocomputing*, 397:292–308, 2020. doi: 10.1016/j.neucom.2019.10.110. URL <https://doi.org/10.1016/j.neucom.2019.10.110>.
- Chen Xu and Yao Xie. Conformal prediction interval for dynamic time-series. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 11559–11569. PMLR, 2021. URL <https://proceedings.mlr.press/v139/xu21h.html>.
- Chen Xu and Yao Xie. Conformal prediction for time series. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(10):11575–11587, 2023. doi: 10.1109/TPAMI.2023.3272339. URL <https://doi.org/10.1109/TPAMI.2023.3272339>.
- Margaux Zaffran, Olivier Féron, Yannig Goude, Julie Josse, and Aymeric Dieuleveut. Adaptive conformal predictions for time series. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 25834–25866. PMLR, 2022. URL <https://proceedings.mlr.press/v162/zaffran22a.html>.
- Jiawen Zhang, Xumeng Wen, Zhenwei Zhang, Shun Zheng, Jia Li, and Jiang Bian. ProbTS: Benchmarking point and distributional forecasting across diverse prediction horizons. In *Advances in Neural Information Processing Systems*, volume 37 of *Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=ZvGk0h7e2N>.